

Computer Graphics Project 1 Report

Ray Tracing and Path Tracing

Youssef Seghrouchni

1 Introduction

This report presents the renderer I progressively implemented during the first four practical sessions. Starting from the skeleton code provided in class, I added the main components of a simple ray tracer and then extended it into a small path tracer. The final program supports sphere intersections, direct lighting, hard shadows, mirror materials, indirect lighting, Monte Carlo sampling, anti-aliasing, triangle mesh rendering from an OBJ file, and BVH acceleration.

The scene is a Cornell-box-style room built from large spheres used as walls, floor, and ceiling. The camera center is fixed at $(0, 0, 55)$, the light source is placed at $(-10, 20, 40)$ with intensity 3×10^7 , and the field of view is 60° . Output colors are gamma-corrected with $\gamma = 2.2$.

Rather than presenting the work strictly lab by lab, I organize this report by implementation steps, which better matches the progression of the code.

2 Basic ray tracing: camera rays, sphere intersections, direct lighting

The first step was to generate one primary ray per pixel. For a pixel (i, j) in an image of width W and height H , I used the standard pinhole camera model:

$$u_{ij} = \left(j - \frac{W}{2} + \frac{1}{2}, \frac{H}{2} - i - \frac{1}{2}, -\frac{W}{2 \tan(\alpha/2)} \right),$$

followed by normalization.

I then implemented ray-sphere intersection. If the ray is

$$X(t) = O + tu,$$

and the sphere has center C and radius R , then the hit condition is

$$\|X(t) - C\|^2 = R^2.$$

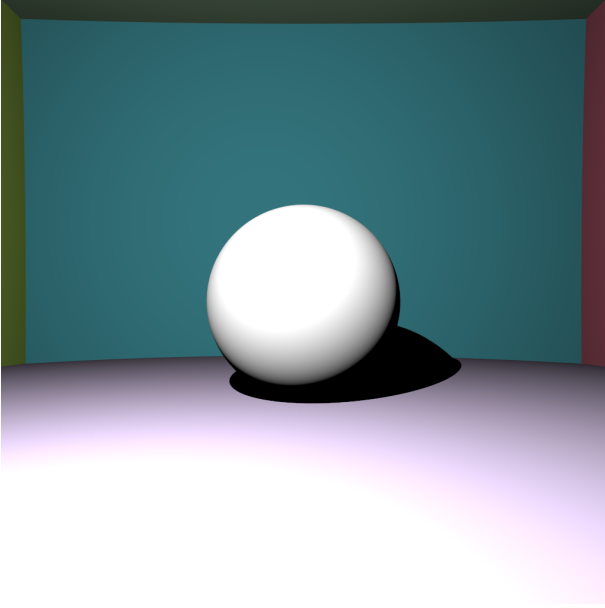
This gives a quadratic equation in t , and the smallest non-negative root is kept. Once t is known, the hit point and normal are

$$P = O + tu, \quad N = \frac{P - C}{\|P - C\|}.$$

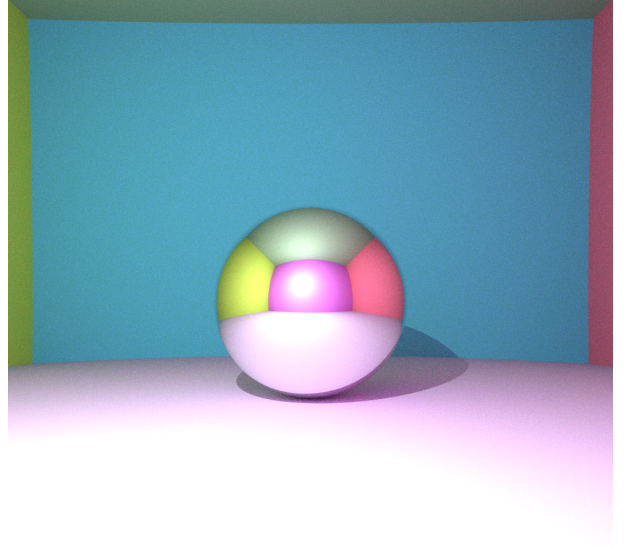
After this, I implemented scene traversal by testing all objects and keeping the closest valid hit. Direct illumination is modeled with a Lambertian BRDF:

$$L_{\text{direct}} = \frac{I}{4\pi d^2} \frac{\rho}{\pi} \max(0, \langle N, \omega_l \rangle),$$

where ρ is the albedo and ω_l is the normalized direction toward the light. Hard shadows are obtained by sending a shadow ray from $P + \varepsilon N$ toward the light source.



(a) Diffuse sphere with direct lighting and shadows.



(b) Mirror material reflecting the colored walls.

Figure 1: First rendering results.

For mirror materials, I used the usual reflection formula

$$r = d - 2(d \cdot N)N,$$

and recursively traced the reflected ray.

3 Path tracing: indirect lighting, Monte Carlo sampling, anti-aliasing

The next step was to extend the renderer from direct lighting to indirect lighting. For diffuse surfaces, the outgoing radiance is

$$L_o(x, \omega_o) = \frac{\rho}{\pi} \int_{\Omega} L_i(x, \omega_i) \langle \omega_i, N \rangle d\omega_i.$$

Instead of evaluating this integral analytically, I estimated it by Monte Carlo integration. At each diffuse hit point, I sampled one random direction in the hemisphere around the normal and recursively traced the corresponding ray.

The hemisphere directions were sampled using cosine-weighted sampling. Given two uniform random variables $r_1, r_2 \in [0, 1)$, I used

$$x = \cos(2\pi r_1) \sqrt{1 - r_2}, \quad y = \sin(2\pi r_1) \sqrt{1 - r_2}, \quad z = \sqrt{r_2}.$$

This local direction was then transformed into world coordinates using an orthonormal basis built around the normal.

Since indirect lighting introduces noise, I averaged several random rays per pixel. This same sampling loop was also used for anti-aliasing, by slightly perturbing the camera ray within the pixel area before normalization.

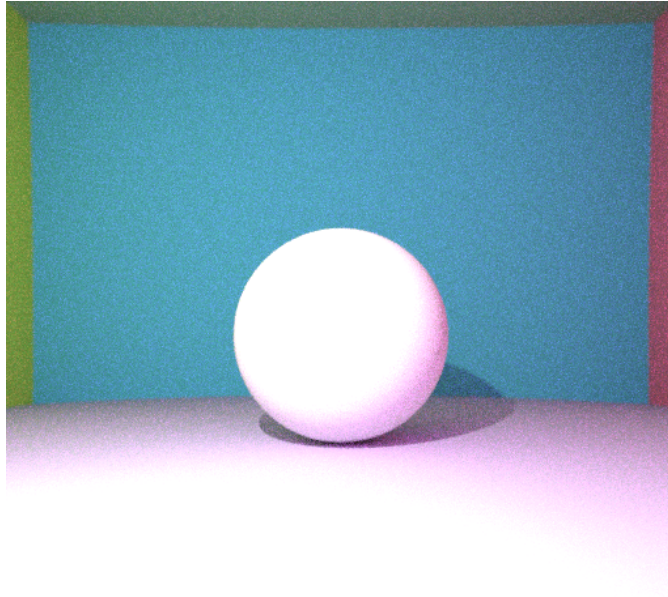


Figure 2: Sphere scene rendered with indirect lighting and Monte Carlo sampling.

Compared to the direct-lighting stage, the image becomes softer and more realistic: dark regions are partially illuminated by light bouncing from the surrounding walls and floor, while the random sampling introduces the expected path tracing noise.

4 Triangle meshes

Once the sphere-based renderer was working, I added support for triangle meshes loaded from an OBJ file. The model used in the project is a cat. After loading, I applied a scale and translation so that the mesh fits correctly inside the room.

For triangle intersection, I implemented a ray-triangle test based on barycentric coordinates. If a triangle has vertices A , B , and C , with

$$e_1 = B - A, \quad e_2 = C - A,$$

then a ray-triangle hit is accepted only if the barycentric coordinates remain inside the triangle and the distance t is positive. Among all valid hits, the closest one is kept.

This step extended the renderer from analytic primitives to general geometry, but performance became much worse, since each ray potentially had to test thousands of triangles.

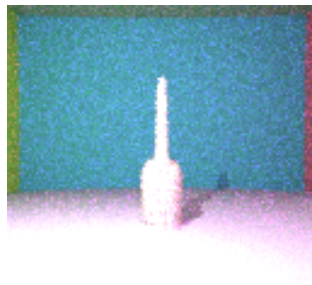


Figure 3: First cat mesh render.

5 BVH acceleration

To make the mesh rendering practical, I implemented a Bounding Volume Hierarchy. Each BVH node stores an axis-aligned bounding box around a subset of triangles, together with child indices or a triangle range if the node is a leaf.

The tree is built recursively. For a given range of triangles, I first compute the bounding box, then choose the longest axis of that box, and split the triangles according to the barycenter of each triangle along this axis. If the range becomes small enough, the node becomes a leaf. During rendering, a ray first tests the node bounding box; if the box is missed, the entire subtree is skipped. Otherwise, traversal continues recursively.

This drastically reduces the number of triangle tests, especially for rays that do not hit the cat at all or only intersect a small part of it.

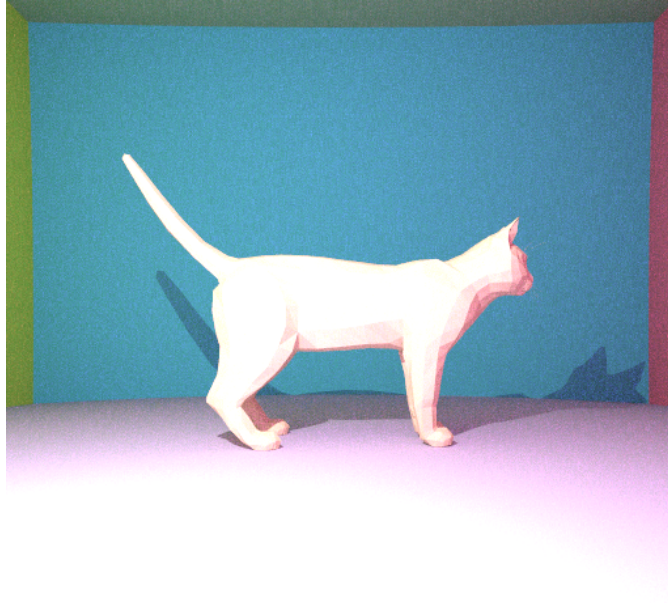


Figure 4: Final cat render with BVH acceleration and global illumination.

6 Rendering times

The precise rendering time depends on the resolution, the number of samples per pixel, and the recursion depth. The values below are only indicative orders of magnitude for comparing the different stages of the implementation.

Scene	Resolution	SPP	Triangles	Time
Sphere, direct lighting + shadows	512×512	1	–	~ 1–2 s
Sphere, indirect lighting	512×512	32	–	~ 20–35 s
Cat mesh without BVH / first mesh stage	128×128	16	3954	~ 15–25 s
Cat mesh with BVH	512×512	64	3954	~ 5–8 s

Table 1: Indicative rendering time summary.

These values are coherent with the main trend observed during the project: adding global illumination increases the cost, triangle meshes are significantly more expensive than spheres, and BVH acceleration is essential to make the final render practical.

7 Conclusion

This project progressively transformed a basic ray tracer into a small path tracer capable of rendering both analytic primitives and a triangle mesh. The most important stages were the implementation of direct illumination and shadows, recursive indirect lighting, Monte Carlo sampling for anti-aliasing and global illumination, triangle mesh support, and finally BVH acceleration.

The final cat scene is a good summary of the whole work: it combines geometry, lighting, sampling, and efficiency in a single renderer. More generally, the project made clear that obtaining a good image is not only a matter of implementing the right formulas, but also of organizing the computation carefully enough to keep render times reasonable.